Protokoll

Datum	Montag, 26.10.2026
Uhrzeit	Ganztag (ca. 08:00 – 18:45 Uhr, mehrere Einheiten)
Ort	zu Hause
Teilnehmer	Julian (Einzelarbeit)
Protokollführer	Claude (KI-Werkzeug)
Thema	Mehrsprachigkeit (DE/EN/FR) und Light-Theme DAW · Light-Theme-Nachbesserung · Passwort-Wiederherstellung Catalogue
Arbeitspakete (PSP)	–
Hilfsmittel	Claude Code (KI), PyCharm, Git, GitHub

Agenda

1. Konzeptionsphase: Ausarbeitung des besten Ansatzes für Sprachumschalter und Theme-Wechsel
2. Implementierung: Übersetzungsmodul, Theme-System, Settings-Dialog
3. Commit und Push auf GitHub
4. Light-Theme-Nachbesserung: verbleibende schwarze Bereiche beheben (mehrere Iterationen)
5. Passwort-Wiederherstellung für Catalogue Manager konzipieren und implementieren
6. Protokollerstellung

Verlauf und Ergebnisse

TOP 1: Konzeption

Ausgangslage: Der `daw_light_version`-Zweig des Projekts war bisher nur auf Englisch verfügbar. Der `Music_catalogue_manager_v.2` bietet dagegen bereits Deutsch, Englisch und Französisch über ein selbst entwickeltes Übersetzungssystem (Custom-Dict in `translations.py`, Cookie-basierte Persistenz).

Analyse: Beide Teilprojekte wurden auf Framework, Theming-Ansatz und i18n-Mechanismus untersucht. Der DAW verwendet PySide6 (Qt für Python); das gesamte Stylesheet war ein einziges, zur Ladezeit ausgewertetes f-String-Literal in `gui_windows.py`.

Designentscheidungen:

Entscheidung	Gewählt	Begründung
i18n-Mechanismus	Custom Python-Dict (analog zum Katalog)	Konsistenz über beide Teilprojekte; kein zusätzliches Tooling (lupdate/lrelease) nötig
Quellsprache	Englisch (Fallback EN)	DAW war bisher nur auf EN; Englisch ist die natürliche Ausgangsbasis
Reihenfolge der Sprachen im Umschalter	DE → EN → FR	Entspricht der Reihenfolge im `Music_catalogue_manager_v.2` (explizite Vorgabe)
Persistenz	`QSettings("SBS", "SynthMaster")`	Qt-native, keine externe Abhängigkeit; plattformübergreifend (macOS Plist / Windows Registry)
Theme-Architektur	`build_stylesheet(palette)` Factory-Funktion	Ermöglicht Laufzeit-Theming ohne Code-Duplikation; ein einziger Ort für Farblogik
Light-Theme-Stil	Professioneller Studio-Look (soft-white Basis, entsättigte Akzentfarben)	Lesbar bei Tageslicht; behält dieselben Farbrollen (cyan/pink/lime)
UI-Platzierung	⚙ SETTINGS-Button links-unten in der Statusbar	Beide Einstellungen (Sprache + Theme) an einem Ort; untere linke Ecke ist in DAWs ungenutzt; `QStatusBar.addWidget()` für links
Änderungsverhalten	Sofort / Live (kein OK/Apply)	Entspricht dem Verhalten des Katalogs beim Sprachwechsel

TOP 2: Implementierung

Neue Datei: `daw_light_version/core/translations.py`

- 28 Übersetzungsschlüssel (Englisch → Deutsch, Französisch): Toolbar-Buttons, Statusbar-Buttons, Dock-Titel, Settings-Dialog-Texte
- `LANGUAGES = ["de", "en", "fr"]` — feste Reihenfolge analog zum Katalog
- Exportiert: `translate(text, lang)`, `LANGUAGES`, `LANGUAGE_LABELS`

Geänderte Datei: `daw_light_version/core/gui_windows.py`

Änderung	Details
`LIGHT_PALETTE`	Neues Farbschema: `#F0F2F5` Hintergrund, `#1A1F2E` Text, `#0077CC` Primärblau etc.
`_hex_to_rgb()` + `build_stylesheet(p)`	Das bisherige `STYLESHEET = f"""..."""` wird durch eine Fabrikfunktion ersetzt, die jeden `rgba()`-Wert aus der Palette berechnet. `STYLESHEET = build_stylesheet(C)` bleibt als Rückwärtskompatibilitäts-Alias.
`SettingsDialog`	Neuer `QDialog` mit `Qt.Tool | Qt.FramelessWindowHint`; DE/EN/FR-Schaltergruppe (CheckButton-Group), Dark/Light-Toggle; emittiert `language_changed` und `theme_changed` als Qt-Signale; übersetzt sich selbst live beim Sprachwechsel.
`MainWindow.__init__`	Liest `QSettings` beim Start; wendet Palette via `QApplication.instance().setStyleSheet()` an (app-weit, nicht nur am Hauptfenster).
`_setup_toolbar()`	Alle 13 Toolbar-Buttons jetzt als `self._tb_*` Instanzattribute gespeichert.
`_setup_transport()`	Dock als `self._transport_dock` gespeichert.
`_setup_mixer()`	Dock als `self._mixer_dock` gespeichert.
`_setup_status_bar()`	Settings-Button als erstes Widget mit `sb.addWidget()` (links); alle anderen Buttons als `self._sb_*` gespeichert.
`retranslate_ui()`	Aktualisiert alle gespeicherten Widget-Texte auf die gewählte Sprache (13 Toolbar + 5 Statusbar + 5 Dock-Titel).
`_apply_theme()`	Ruft `QApplication.instance().setStyleSheet(build_stylesheet(palette))` auf.
`_on_settings()`	Öffnet den `SettingsDialog` direkt über dem Settings-Button (anchored positioning).

Geänderte / neue Dateien:

- `daw_light_version/core/translations.py` — neu erstellt
- `daw_light_version/core/gui_windows.py` — 434 Zeilen netto hinzugefügt

Syntaxprüfung: `python3 -c "from core import gui_windows"` — erfolgreich, kein Fehler.

TOP 3: Commit und Push (Einheit 1)

- Commit `48fb519` — `feat(daw): add DE/EN/FR language switching and light theme`
- Gepusht auf `https://github.com/JulianWIAI/graduation_project.git`

TOP 4: Light-Theme-Nachbesserung (Einheit 2 + 3)

Nach dem ersten Test waren mehrere Bereiche im Light-Theme noch dunkel. Die Probleme wurden in drei weiteren Commits iterativ behoben:

Iteration 1 — `1675f12`**:** Alle hardkodierten `background:{C['deep/abyss']}` aus Toolbar-Buttons, Statusbar-Buttons, TransportBar, EffectsPanel, MixerStrip und Mixer-Containern entfernt. Sprachbuttons von `setFixedSize(42,28)` auf `setFixedHeight(28)+setMinimumWidth(52)` geändert (waren abgeschnitten).

Iteration 2 — `bee05a1`**:** Track-View-Canvas (`PianoRollWidget`, `TrackArrangeView`) blieb schwarz, weil `paintEvent()` zur Laufzeit `C["abyss"]`/`C["void"]` liest. Lösung: `_apply_theme()` mutiert nun `C` in-place via `C.update(palette)` vor dem Stylesheet-Wechsel. `DARK_PALETTE = dict(C)` als eingefrorene Kopie für den Rückwechsel hinzugefügt.

Iteration 3 — `e4b29dd`**:** Restliche dunkle Bereiche bereinigt (15 Dateien):

- In `gui_windows.py`: 6 Dialog-`setStyleSheet(STYLESHEET)` → `setStyleSheet(build_stylesheet(C))` (dynamisch); 5 Piano-Roll-Layoutcontainer-Hintergründe entfernt; Panic-Button `background:#1A0000` entfernt
- `auto_mastering_dialog.py`, `export_dialog.py`, `gm_defaults_dialog.py`, `channel_rack.py`: eigene `_STYLESHEET`-Aufrufe entfernt
- `fx_rack_widget.py`, `fx_slot_widget.py`, `grid_settings_panel.py`, 10 FX-Plugin-Dateien: dunkle Container-Hintergründe entfernt

TOP 5: Passwort-Wiederherstellung Catalogue Manager (Einheit 3)

Commit `20d802f` — `feat(catalogue): two-layer password recovery`

Schicht 1 — Sicherheitsfrage (Selbstbedienung):

- Registrierung: 2 neue Felder (Frage + Antwort, Antwort bcrypt-gehasht, kleingeschrieben)
- Flow: `/forgot-password` → `/forgot-password/<email>` (Frage anzeigen) → `/reset-password/<email>/<token>` (neues Passwort)
- 3-Versuchs-Sperre analog zur bestehenden Login-Sperre
- 4 neue Templates: `forgot_password.html`, `forgot_password_question.html`, `reset_password.html`, `change_password.html`

Schicht 2 — Admin-Temporärpasswort (Fallback):

- Einstellungen-Seite: neue Sektion „Benutzerverwaltung“ mit Nutzer-Tabelle
- „Zurücksetzen“-Button generiert 8-stelliges Hex-Passwort, zeigt es einmalig per Flash
- `must_change_pw = 1` → Pflicht zur Passwortänderung beim nächsten Login

DB-Migration: 3 neue Spalten (`security_question`, `security_answer_hash`, `must_change_pw`), rückwärtskompatibel via `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`

Übersetzungen: ~20 neue Einträge in `translations.py` (DE → EN / FR)

TOP 6: Light-Theme-Nachbesserung (Einheit 4) – Master, Telemetrie, Dialog-Übersetzungen

Nach Sichtung von Screenshots aus Einheit 3 wurden vier weitere Probleme identifiziert und behoben:

Problem 1 — MasterBusChannel-Hintergrund dunkel: `master_bus_channel.py` hat eine eigene `_C`-Palette und setzt in `__init__` `background:{_C['abyss']}` als Widget-Stylesheet. Da Widget-Stylesheets Vorrang vor dem app-weiten QSS haben, blieb der Master-Streifen immer dunkel. Lösung: `background:` aus dem `setStyleSheet`-Aufruf entfernt; Border/Border-left/Radius/Margin bleiben.

Problem 2 — Telemetrie-Container dunkel: `telemetry_manager.py`, `_build_bands_container()`: lokale Variable `_BG = "#0a0a14"` wurde via `container.setStyleSheet(f"background:{_BG};")` gesetzt. Lösung: Diese Zeile entfernt; app-weites QSS übernimmt den Hintergrund automatisch.

Problem 3 — Fenster/Dialoge in falscher Sprache: `AutoMasterDialog` (`auto_mastering_dialog.py`) hatte alle Labels/Titel hartcodiert auf Englisch. Das Fenster wurde stets ohne Sprachparameter geöffnet. Lösung:

- `translations.py`: 17 neue Schlüssel für den Dialog (Fenstertitel, Gruppenboxen, Formular-Labels, Buttons, Platzhaltertexte).
- `auto_mastering_dialog.py`: `__init__` nimmt jetzt `lang="en"`; alle sichtbaren Strings nutzen `_tr(…, self._lang)`.
- `gui_windows.py`, `_on_auto_master()`: `AutoMasterDialog(…, lang=self._lang)`.

Problem 4 — „DS INSTRUMENT"-Dock-Tab immer auf Englisch: `DsPresetPanel` setzt seinen Titel im eigenen `__init__` auf den festen String `"DS INSTRUMENT"`. `retranslate_ui()` aktualisierte diesen Dock bisher nicht. Lösung: `translations.py` Eintrag `"DS INSTRUMENT"` ergänzt; `retranslate_ui()` ruft jetzt `self._ds_panel.setWindowTitle(_tr("DS INSTRUMENT", L))` auf.

Geänderte Dateien:

- `daw_light_version/core/translations.py` — 17 neue Schlüssel
- `daw_light_version/core/master_bus_channel.py` — `background:` aus `setStyleSheet` entfernt
- `daw_light_version/core/telemetry_manager.py` — `container.setStyleSheet(background)` entfernt
- `daw_light_version/core/auto_mastering_dialog.py` — `lang`-Parameter + `_tr()`-Aufrufe
- `daw_light_version/core/gui_windows.py` — `retranslate_ui` + `_on_auto_master` angepasst

TOP 7: Weitere Nachbesserungen (Einheit 5)

Nach Sichtung weiterer Screenshots wurden drei zusätzliche Probleme identifiziert und behoben:

Problem 1 — MasterBusChannel: Innere Widgets bleiben dunkel im Light-Theme: Die Unterwidgets des Master-Strips (AUDITION-Buttons PREV/STRM, Gain-Fader-Groove, LIMITER-Button, Ceiling-Spinbox) verwenden alle das eingefrorene Modul-`_C`-Dict aus `master_bus_channel.py`. Das frühere Fix hat nur den äußeren Container-Hintergrund entfernt, nicht die Kinder-Stylesheets. Lösung: `MasterBusChannel.set_theme(palette)` hinzugefügt — mutiert `_C` in-place und ruft `setStyleSheet()` auf alle Unterwidgets sowie `self._meter.update()` auf. `_apply_theme()` in `gui_windows.py` ruft nun `self._master_bus_channel.set_theme(palette)` auf.

Problem 2 — Telemetrie-Widget-Hintergründe (Verbesserung vorheriger Fix): Alle 5 Telemetrie-Canvas-Widgets (`TelemetryWaveformWidget`, `TelemetryBandWidget`, `TelemetryChromaWidget`, `TelemetryWaterfallWidget`, `TelemetryHpssWidget`) haben eigene eingefrorene `_BG = QColor(...)` Modulkonstanten in `paintEvent`. Lösung: `set_bg(color: QColor)` / `self._bg`-Instanzmuster in allen 5 Widgets; `TelemetryManager.set_theme(palette)` ruft `set_bg()` auf alle auf; `_apply_theme()` ruft `self._telemetry.set_theme(palette)` auf. *Hinweis: Telemetrie-Panels behalten bewusst dunklen Hintergrund — Oszilloskop- und Spektrum-Anzeigen sind in der DAW-Praxis stets dunkel (wie FL Studio, Ableton), da die farbigen Signallinien auf weißem Hintergrund nicht lesbar sind.*

Problem 3 — Auto-Assign Instruments / KI-Mix-Dialog noch auf Englisch: `_on_randomize_instruments()` und `_on_ai_mix()` verwendeten hartcodierte englische QMessageBox-Titel und -Texte sowie ein eingefrorenes lokales `_C`-Dict im AI-Mix-Dialog. Lösung:

- `translations.py`: 29 neue Schlüssel (AutoMaster, AI Mix, Auto-Assign, DS INSTRUMENT)
- `_on_randomize_instruments()`: alle 3 sichtbaren Strings nutzen `_tr()`; dynamische Ergebnisnachricht mit `.format(n=..., total=...)`
- `_on_ai_mix()`: lokales `_C` durch live `C` ersetzt; Fenstertitel, Label und Buttons nutzen `_tr()`

Commits dieser Einheit: `10c73c8`, `6ac3c18`, `f7d7b11`

Geänderte Dateien (Einheit 5):

- `daw_light_version/core/master_bus_channel.py` — `set_theme(palette)` hinzugefügt
- `daw_light_version/core/gui_windows.py` — `_apply_theme` + AI-Mix-Dialog + Auto-Assign
- `daw_light_version/core/translations.py` — 29 neue Schlüssel gesamt
- `daw_light_version/core/telemetry_manager.py` + 5 Telemetrie-Widget-Dateien — `set_bg()`-Muster

TOP 8: MasterBusChannel – transparente Hintergründe (Einheit 6)

Visueller Test des Master-Sliders im Clair-Theme (Screenshot 19:12 Uhr) zeigte: AUDITION-Buttons PREV/STRM, Fader-Groove und LIMITER-Button weiterhin dunkel, obwohl `set_theme(palette)` korrekt aufgerufen wurde.

Ursache: `set_theme()` mutiert `_C` und ruft `setStyleSheet()` erneut auf — jedoch reagieren die Buttons auf den Aufruf zur Laufzeit nicht sichtbar, da Qt den Stil bereits gecacht hatte. Das grundlegende Problem: Alle `background:{_C['deep']}` in den Widget-Stylesheets sind bei Initialisierung eingefrorene Dunkelwerte.

Lösung: Dieselbe Strategie wie bei allen anderen Buttons im DAW — `background:` vollständig entfernen, damit das app-weite QSS den Hintergrund übernimmt:

- `_mode_btn_style()` (inaktiver Zustand): `background: {_C['deep']}` → `background: transparent`
- Fader `setStyleSheet` in `__init__`: `background:{_C['deep']}` aus Groove und Sub-Page entfernt
- LIMITER-Button `setStyleSheet` in `__init__`: `background:{_C['deep']}` aus Normal- und Disabled-Zustand entfernt
- Ceiling-Spinbox `setStyleSheet` in `__init__`: `background:{_C['deep']}` entfernt

Ergebnis (Screenshot 19:27 Uhr): AUDITION-Buttons, Fader und LIMITER-Bereich korrekt hell im Clair-Theme. VU-Meter-Balken bleiben bewusst dunkel (Messgeräte-Konvention).

Commit: `b277ae2`

Geänderte Datei: `daw_light_version/core/master_bus_channel.py`

Aufgaben und nächste Schritte

Aufgabe	Status	Verantwortlich
DAW starten und Settings-Dialog testen	✓ erledigt	Julian
Dock-Titel auf korrekte Übersetzung prüfen	✓ erledigt	Julian
Light-Theme: verbleibende schwarze Bereiche beheben	✓ erledigt (6 Iterationen)	Julian
Dialog-Sprache (AutoMaster, DS INSTRUMENT, AI Mix, Auto-Assign)	✓ erledigt	Julian
Master-Slider im Clair-Theme korrekt	✓ erledigt (visuell bestätigt 19:29 Uhr)	Julian
Passwort-Wiederherstellung visuell testen (Registrierung, Vergessen-Flow, Admin-Reset)	offen	Julian

Aufgabe	Status	Verantwortlich
Piano-Roll-Noten und Piano-Tasten im Light-Theme auf Lesbarkeit prüfen	offen	Julian

Dieses Protokoll wurde maschinell erstellt und ist ohne Unterschrift gültig.

gez. Julian